Politecnico di Milano
Numerical Methods for Partial Differential Equations — A.Y. 2025/2026
Prof. Alfio Quarteroni — Dr. Michele Bucelli

Preconditioning Heterogeneous Diffusion Problems

Nada Elsayed	10998973	Wu Cao	11036000
Fude Zhou	10679500	Elena Nuttini	10683877

Abstract

This work studies iterative solution strategies for a three-dimensional elliptic equation whose diffusion coefficient jumps by as much as five orders of magnitude. The equation is discretized with continuous trilinear finite elements and solved with conjugate gradients using the parallel deal.II/Trilinos stack. Five choices are compared: no preconditioner, Jacobi, SSOR, ILU, and algebraic multigrid (AMG). A manufactured solution verifies the expected second-order L^2 and first-order H^1 convergence. A 300-case sweep then varies coefficient contrast, mesh resolution, and MPI process count. The measurements show that AMG is the only tested method that remains consistently robust: in the hardest serial case it reduces the iteration count from 16,067 to 31 and the estimated condition number from 1.05×10^7 to 18.5. The report also separates setup and solve cost, assesses mesh dependence and strong scaling, and explains why low iteration count, low wall time, and good parallel speedup are distinct goals.

1 Introduction

Large coefficient jumps arise in composite materials, porous media, and subsurface flow. They make the finite-element stiffness matrix increasingly ill-conditioned, so an otherwise appropriate Krylov method may require thousands of iterations. The project brief asks not only whether a preconditioner accelerates a single solve, but whether it remains effective as heterogeneity and problem size grow and whether it uses parallel resources efficiently.

Our contribution is a reproducible comparison in which all methods see the same matrix, right-hand side, stopping rule, geometry, and hardware environment. The central question is: *which preconditioner best balances robustness, total time, optimality, and scalability for this problem?*

2 Mathematical formulation

Let $\Omega = (0,1)^3$ and let $B = \bigcup_{i=1}^{12} B_i$ be a fixed union of spherical inclusions. For $p > 0$, define

$$\mu_p(\boldsymbol{x}) = \begin{cases} 10^p, & \boldsymbol{x} \in B, \\ 1, & \boldsymbol{x} \in \Omega \setminus B. \end{cases} \tag{1}$$

We look for $u : \overline{\Omega} \to \mathbb{R}$ satisfying

$$-\nabla \cdot (\mu_p \nabla u) = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega, \qquad f \equiv 1. \tag{2}$$

The contrast is $\mu_{\max}/\mu_{\min} = 10^p$; the experiments use $p = 1, \ldots, 5$.

Multiplying eq. (2) by $v \in H_0^1(\Omega)$, integrating, and using the zero trace of v eliminates the boundary term. The weak problem is therefore: find $u \in V = H_0^1(\Omega)$ such that

$$a(u,v) = \ell(v) \quad \forall v \in V, \qquad a(u,v) = \int_\Omega \mu_p \nabla u \cdot \nabla v \, \mathrm{d}x, \quad \ell(v) = \int_\Omega f v \, \mathrm{d}x. \tag{3}$$

The discontinuities in μ_p do not invalidate this formulation: only essential boundedness and a positive lower bound are required. Flux $\mu_p \nabla u \cdot n$ is transmitted across an inclusion interface, while u remains continuous in the finite-element space. Because the mesh does not conform to the spheres, coefficient values are sampled at quadrature points; thus the geometric interface is represented at quadrature resolution rather than by curved element faces.

This detail matters when interpreting refinement studies. Refinement simultaneously improves the approximation of u and of the discontinuous coefficient geometry. Consequently, the main study focuses on solver behavior; approximation accuracy is checked separately with a smooth homogeneous manufactured problem.

3 Well-posedness and conditioning

For $u, v \in V$, Cauchy–Schwarz gives

$$|a(u,v)| \le \mu_{\max} |u|_{H^1(\Omega)} |v|_{H^1(\Omega)}. \tag{4}$$

Since $\mu_p \ge 1$ almost everywhere and Poincaré's inequality holds on $H_0^1(\Omega)$,

$$a(v,v) \ge |v|^2_{H^1(\Omega)} \ge \frac{1}{1+C_P^2} \|v\|^2_{H^1(\Omega)}. \tag{5}$$

The load is bounded by $|\ell(v)| \le \|f\|_{L^2}\|v\|_{L^2} \le C_P \|f\|_{L^2}|v|_{H^1}$. Lax–Milgram therefore yields a unique weak solution for every tested p [1].

After discretization, the symmetric positive-definite stiffness matrix permits conjugate gradients (CG). In exact arithmetic, its energy-norm error obeys

$$\frac{\|e_k\|_A}{\|e_0\|_A} \le 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k, \tag{6}$$

so iteration count is governed by the spectrum rather than matrix size alone [2]. For elliptic finite elements one informally expects conditioning to worsen with both h^{-2} and coefficient contrast. The data confirm this interaction: for refinement 5, the unpreconditioned condition estimate rises from 1.14×10^3 at $p = 1$ to 1.05×10^7 at $p = 5$.

Preconditioning replaces the system by an algebraically equivalent one with a more favorable spectrum. With a symmetric positive-definite M, the relevant operator is similar to $M^{-1/2} A M^{-1/2}$, although software commonly applies M^{-1} directly. An effective M clusters eigenvalues, remains inexpensive to construct and apply, and preserves favorable communication patterns. These requirements compete: richer approximations to A generally reduce iterations but cost more memory, setup work, and synchronization.

4 Finite-element discretization

From the suitable discrete space:

$$V_h = \{v_h \in C^0(\overline{\Omega}) : v_h|_K \in P_1(K),\ v_h|_{\partial\Omega} = 0\}. \tag{7}$$

Writing $u_h = \sum_{j=1}^{N_h} U_j \phi_j$ and testing with each nodal basis function gives

$$A\boldsymbol{U} = \boldsymbol{b}, \qquad A_{ij} = \int_\Omega \mu_p \nabla\phi_j \cdot \nabla\phi_i \,\mathrm{d}x, \quad b_i = \int_\Omega f\phi_i \,\mathrm{d}x. \tag{8}$$

The assembly uses the deal.II simplex quadrature rule of order two. Since P_1 shape-function gradients are constant on each tetrahedron, the stiffness integrand is integrated exactly when the coefficient is constant over the quadrature neighborhood. Cells cut by an inclusion interface inherit pointwise coefficient samples, which is consistent with the unfitted implementation.

Table 1: Uniform-grid sizes used in the benchmark. The cube is subdivided into 2^r intervals per direction and each Cartesian cell is split into six tetrahedra. The P_1 DoFs are the vertices, so $N_h = (2^r + 1)^3$ includes boundary nodes; homogeneous Dirichlet constraints eliminate boundary values algebraically.

Refinement r	$h = 2^{-r}$	active tetrahedra $6\,8^r$	global DoFs N_h
3	1/8	3,072	729
4	1/16	24,576	4,913
5	1/32	196,608	35,937

The expected approximation rates for smooth u are $\|u - u_h\|_{L^2} = O(h^2)$ and $\|u - u_h\|_{H^1} = O(h)$. The discontinuous production coefficient can reduce regularity, so these textbook rates are verified with $\mu = 1$ rather than claimed for the heterogeneous solution. This separation distinguishes code verification (solving the equations correctly) from the performance experiment (solving difficult systems efficiently).

5 Linear solver and stopping criterion

Every system is solved by CG from the zero vector. The deal.II control object uses a maximum of 20,000 iterations and the absolute tolerance

$$\|\boldsymbol{r}_k\| \le 10^{-10}\|\boldsymbol{b}\|, \qquad \boldsymbol{r}_k = \boldsymbol{b} - A\boldsymbol{U}_k. \tag{9}$$

This relative residual rule is identical across methods, contrasts, refinements, and process counts, making iteration counts directly comparable. Timings use wall clock time and an MPI maximum, so the reported value represents the slowest rank rather than an optimistic rank average.

The program also connects deal.II's CG condition-number estimator. It derives an estimate from the Lanczos tridiagonal generated during CG. This quantity is useful diagnostically, but it is not an independent eigensolve and becomes unavailable when convergence takes only one iteration. We consequently treat negative values in the raw CSV as "not estimated", not as physical condition numbers.

For each (p, r, np) case, mesh creation and assembly happen once. The five solver calls then reuse the same matrix and right-hand side, reset the solution to zero, initialize their own preconditioner, and record

$$t_{\text{total}} = t_{\text{pc setup}} + t_{\text{solve}}. \tag{10}$$

Mesh setup and matrix assembly are reported separately in the raw data and excluded from this solver total. This is appropriate for comparing reusable linear-algebra strategies, but an application that solves only once should add assembly cost; an application with many right-hand sides may amortize setup and favor a more expensive preconditioner.

CG formally requires a symmetric positive-definite preconditioned operator. The selected deal.II/Trilinos wrappers are used in their default configurations, and all 300 recorded solves completed successfully. Default parameters are intentionally retained to compare accessible out-of-the-box choices rather than individually tuned variants.

6 Preconditioners under consideration

Let $A = D + L + U$, with D diagonal and L, U strictly triangular.

Identity. $M = I$ supplies the baseline. It has no setup or application overhead but exposes CG to the full heterogeneity- and mesh-induced spectrum.

Jacobi. $M = D$ scales each unknown by its diagonal entry. Application is pointwise and parallel, but off-diagonal couplings and global low-frequency error remain untreated.

SSOR. In idealized form,

$$M_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}(D + \omega L)D^{-1}(D + \omega U). \tag{11}$$

Forward and backward sweeps represent more local structure than Jacobi, at the price of ordering dependencies that are awkward across MPI subdomains.

ILU. An incomplete factorization constructs $A \approx \widetilde{L}\widetilde{U}$ while discarding fill according to the implementation policy. Triangular applications can be powerful locally but have sequential dependencies and become effectively block-local in distributed memory.

AMG. Algebraic multigrid constructs levels from the matrix graph. A schematic V-cycle is

$$M_{\mathrm{AMG}}^{-1} r \approx S_h r + P A_H^{-1} R(r - A S_h r) + \text{post-smoothing}, \tag{12}$$

where relaxation damps oscillatory error and the coarse correction addresses algebraically smooth error [3]. The implementation enables the elliptic and higher-order-element flags and uses one cycle. AMG has the largest conceptual and setup complexity, but it is the only candidate designed to approximate the inverse across length scales.

The comparison therefore spans a useful hierarchy: no information, diagonal information, local relaxation, local factorization, and multilevel global information. It tests whether added structure is repaid by fewer iterations and lower total time.

7 Parallel implementation

The solver uses `parallel::distributed::Triangulation<3>`. Each MPI rank assembles only its locally owned cells, while locally relevant ghost indices support constraints and finite-element couplings. `TrilinosWrappers::SparsityPattern`, sparse matrix, and distributed vectors retain the ownership partition. Local element matrices and vectors are inserted through affine constraints, then globally compressed with additive semantics.

The implementation sequence is

1. generate and partition the uniformly refined cube;
2. distribute DoFs and impose homogeneous Dirichlet values;
3. create the distributed sparsity pattern, matrix, and vectors;
4. integrate locally owned cells and compress the global objects;
5. initialize one preconditioner, run CG, and collect global-maximum timings;
6. repeat the solve for all five methods without rebuilding A.

Matrix-vector products require halo exchange. Jacobi adds almost no further communication. SSOR and ILU introduce subdomain-sensitive local operations, and AMG adds hierarchy construction, restriction/prolongation, and a coarse problem whose parallelism diminishes as its size decreases. Thus an iteration reduction does not automatically imply ideal strong scaling.

The same global problem is used for $np \in \{1, 2, 3, 4\}$, so the experiment is a *strong* scaling study. Speedup is

$$S(np) = \frac{T(1)}{T(np)}, \qquad E(np) = \frac{S(np)}{np}. \tag{13}$$

At only 35,937 DoFs the finest problem is small for distributed-memory benchmarking. Fixed costs, process launch noise, coarse-grid overhead, and communication can dominate. Results are therefore evidence about this tested regime, not a claim about asymptotic performance on large clusters.

8 Experimental design and reproducibility

The full factorial sweep comprises

$$5\ p\text{-values} \times 3\ \text{meshes} \times 5\ \text{preconditioners} \times 4\ \text{MPI sizes} = 300\ \text{solves}.$$

The Python driver builds the executable, selects an available launcher, stores one log per process count, parses structured output, and writes `result/benchmark_results.csv`. All figures in this report read that CSV directly, making numerical claims traceable to a row of raw data.

Table 2: Controlled factors and recorded responses.

Factor or metric	Values / definition
Coefficient exponent	$p = 1, 2, 3, 4, 5$; contrast 10^p
Mesh refinement	$r = 3, 4, 5$; see table 1
MPI processes	$np = 1, 2, 3, 4$
Preconditioner	identity, Jacobi, SSOR, ILU, AMG
Solver response	iterations, estimated condition number
Time response	preconditioner setup, solve, their sum; seconds
Shared controls	zero initial guess, tolerance eq. (9), same matrix/RHS

To reproduce from a configured course environment:

```
source setup_modules.sh
cmake -S . -B build
cmake --build build -j
./build/elliptic verify
python3 src/script/benchmark_preconditioners.py
latexmk -pdf report.tex
```

The CSV currently committed with the report is the preserved dataset used for analysis. Timing measurements should be regenerated on a new machine rather than compared across machines as if hardware were controlled. Iteration counts are much more portable because they are primarily algorithmic, although MPI-local SSOR/ILU behavior can depend on partitioning.

9 Verification with a manufactured solution

Code verification replaces the heterogeneous coefficient by $\mu = 1$ and chooses

$$u_{\mathrm{ex}}(x, y, z) = \sin(\pi x)\sin(\pi y)\sin(\pi z), \qquad f = 3\pi^2 u_{\mathrm{ex}}. \tag{14}$$

This solution satisfies the homogeneous boundary condition. Errors are integrated with a higher quadrature order than assembly and are reduced globally over the distributed mesh.

Table 3: Manufactured-solution convergence. Rates use successive halving of h.

h	$\|e\|_{L^2}$	rate	$\|e\|_{H^1}$	rate
1/8	5.7462×10^{-3}	–	2.1818×10^{-1}	–
1/16	1.4367×10^{-3}	2.00	1.0905×10^{-1}	1.00
1/32	3.5919×10^{-4}	2.00	5.4524×10^{-2}	1.00

The observed rates exactly match the continuous P_1 expectations. Halving h divides the L^2 error by four and the H^1 error by two. This simultaneously exercises mesh creation, quadrature, load and stiffness assembly, constraints, the distributed solve, ghost-vector construction, and error integration. It does not validate the benchmark's absolute timings or the geometric accuracy of unfitted sphere interfaces; those are separate concerns.

The verification solve uses AMG only to reach the discrete solution efficiently. Because all preconditioners leave eq. (8) unchanged when converged, convergence rates test the discretization rather than rank the preconditioners. Together with the successful completion of all 300 production solves, the study provides a sound basis for analyzing solver performance.

10 Coefficient geometry and solution regime

The twelve inclusions have distinct centers and radii between 0.060 and 0.095. They are distributed throughout the cube rather than aligned in a layer, forcing global error components to interact with many isolated high-conductivity regions. Their geometry is fixed while p changes; this isolates coefficient contrast as the experimental variable.

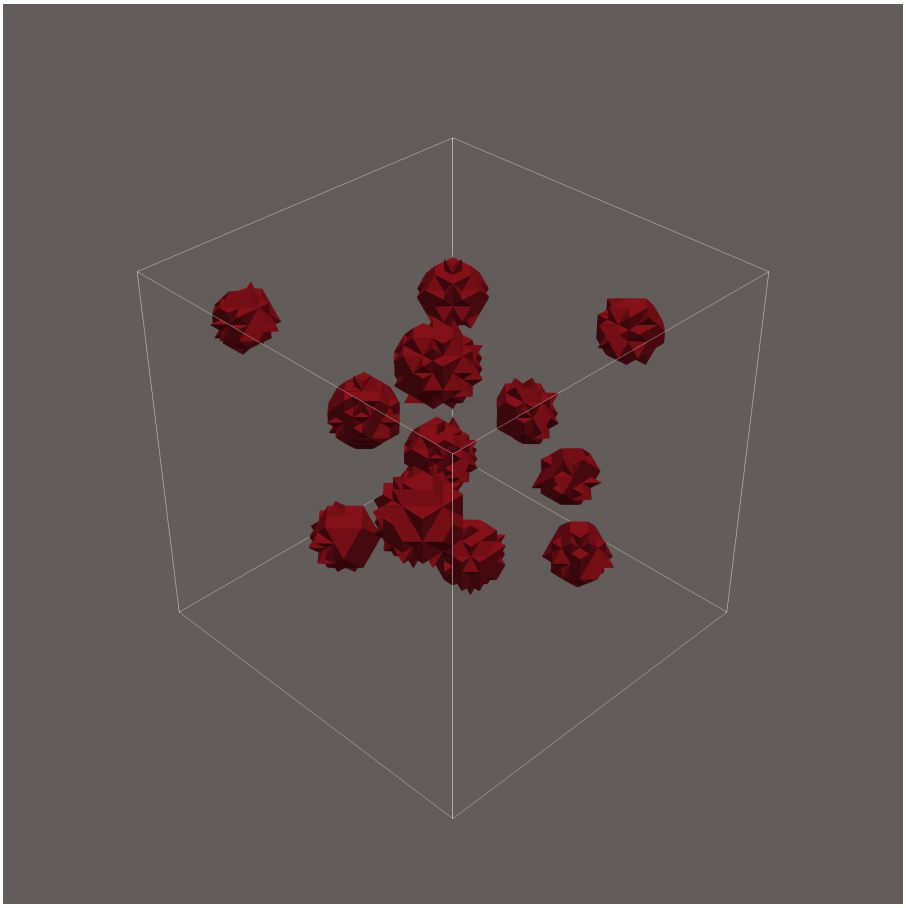


Figure 1: Cellwise visualization of the coefficient field for $p = 5$. Orange inclusions have $\mu = 10^5$ and the background has $\mu = 1$. The apparent faceting comes from evaluation on the uniform tetrahedral mesh.

With $f = 1$, the solution is positive in the interior. High-μ regions penalize gradients more strongly, so the solution tends to become locally flatter inside them while flux continuity couples them to the background. Algebraically, element contributions in the inclusions are scaled by 10^p, creating widely separated energy scales. A diagonal correction can normalize individual rows, but it cannot fully represent the collective modes produced by separated inclusions. This is the physical origin of the preconditioning challenge.

11 Robustness with respect to heterogeneity

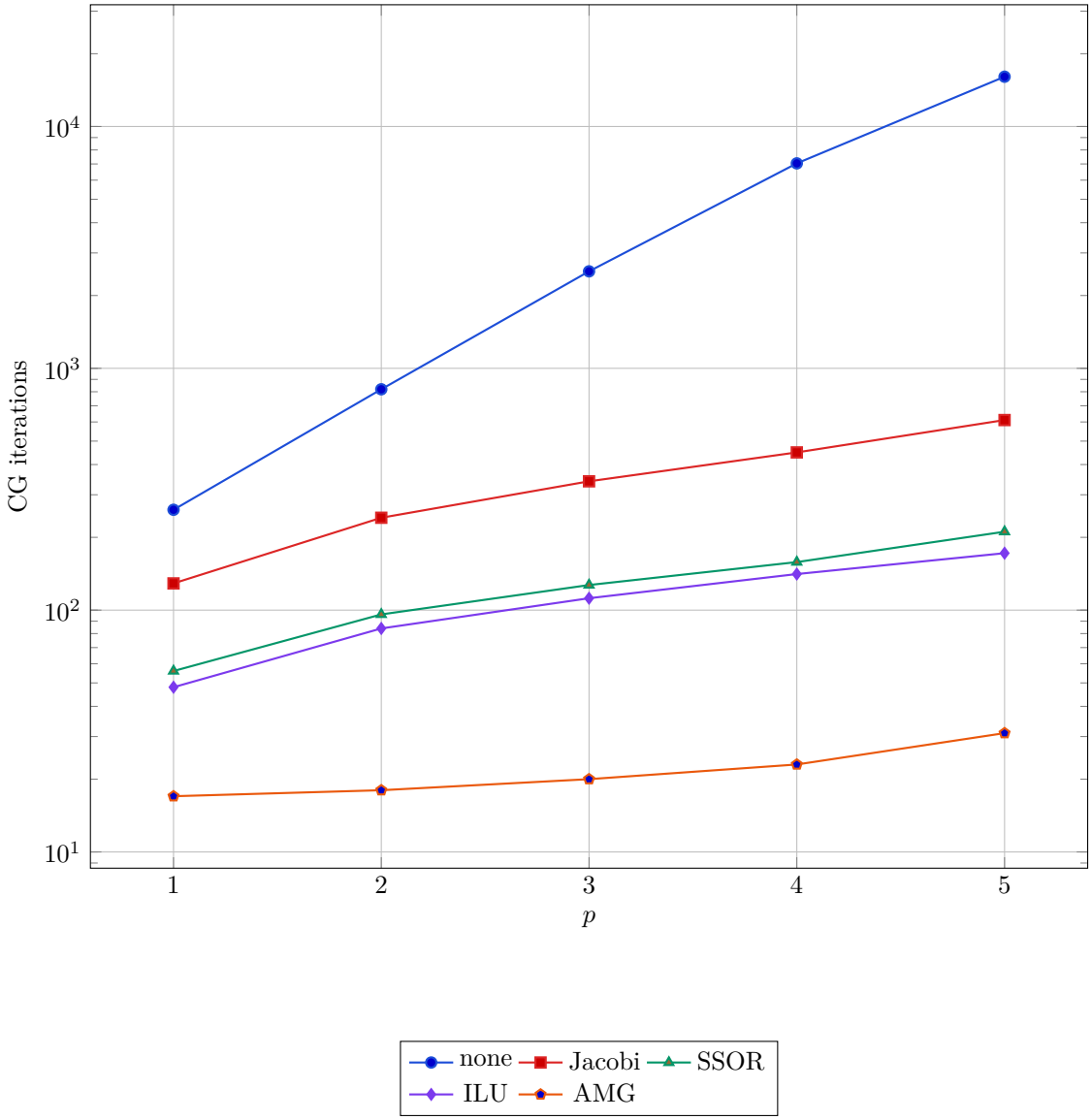


Figure 2: Iteration count against contrast exponent on the finest serial mesh. Data are tabulated in a compact derived file to keep plot filtering transparent.

The baseline grows from 260 iterations at $p = 1$ to 16,067 at $p = 5$, a factor of 61.8. Jacobi improves the hardest case to 611 iterations, SSOR to 211, and ILU to 172, but all still degrade as contrast rises. AMG changes only from 17 to 31 iterations, a factor of 1.82. In this operational sense AMG is robust with respect to the tested heterogeneity, though not perfectly contrast independent.

12 Spectral evidence

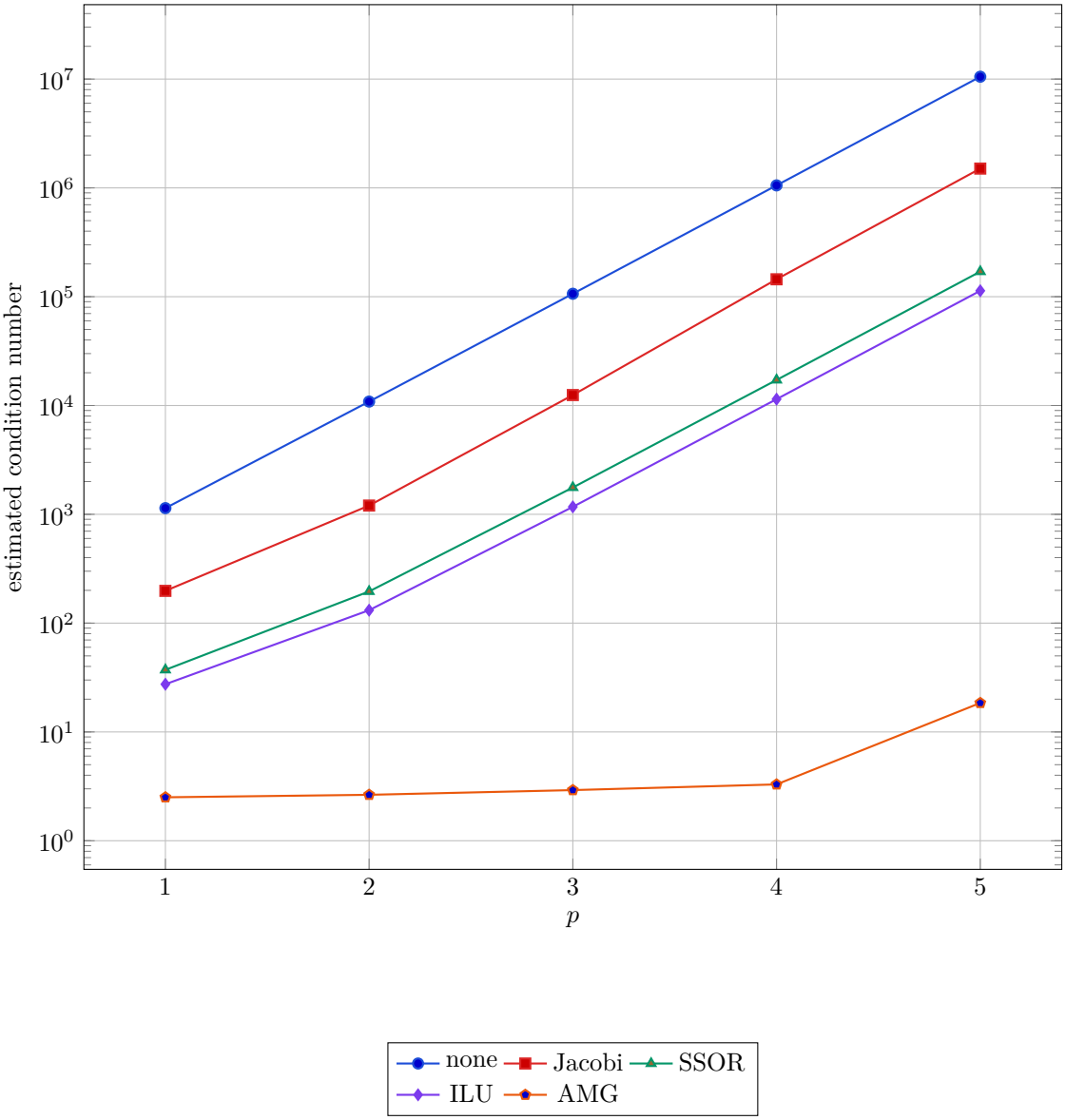


Figure 3: CG/Lanczos condition estimates at $r = 5$, $np = 1$.

At $p = 5$, the estimated condition numbers are 1.05×10^7 (none), 1.50×10^6 (Jacobi), 1.70×10^5 (SSOR), 1.13×10^5 (ILU), and 18.5 (AMG). The ordering mirrors the iteration counts and supports the spectral explanation in eq. (6). AMG's estimate stays between 2.51 and 18.5 over all five contrasts, whereas the other curves inherit a near order-of-magnitude increase for each increment of p.

These are effective preconditioned condition estimates, not exact matrix condition numbers. Their value is comparative: they show that AMG changes the spectrum qualitatively, while diagonal and local approaches mainly reduce its scale.

13 Total time on the hardest serial case

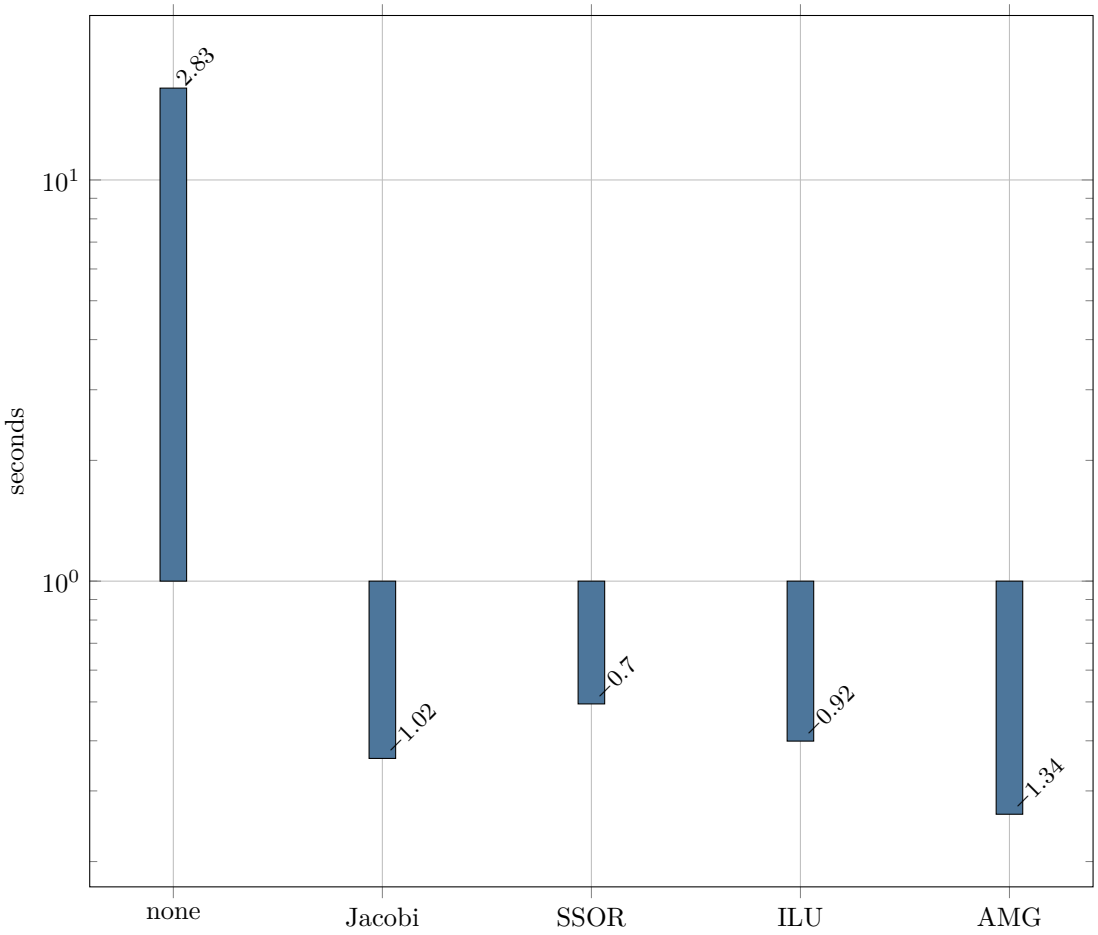


Figure 4: Preconditioner setup plus solve time for $p = 5$, $r = 5$, $np = 1$. Logarithmic scale.

AMG is fastest overall at 0.262 s despite its nonzero setup. Relative to no preconditioner it gives a 64.6× reduction; relative to Jacobi, SSOR, and ILU it is respectively 1.38×, 1.88×, and 1.52× faster. ILU needs fewer iterations than SSOR and also finishes sooner, while Jacobi's very cheap applications offset its much larger iteration count.

Timing very short cases is noisy, which is why the hardest case is most informative. The conclusion is nonetheless clear at the scale of the baseline gap: iteration robustness is large enough to repay AMG construction. For repeated solves with the same matrix, reusing the AMG hierarchy would make its advantage stronger than the one-shot totals shown here.

14 Setup and solve cost decomposition

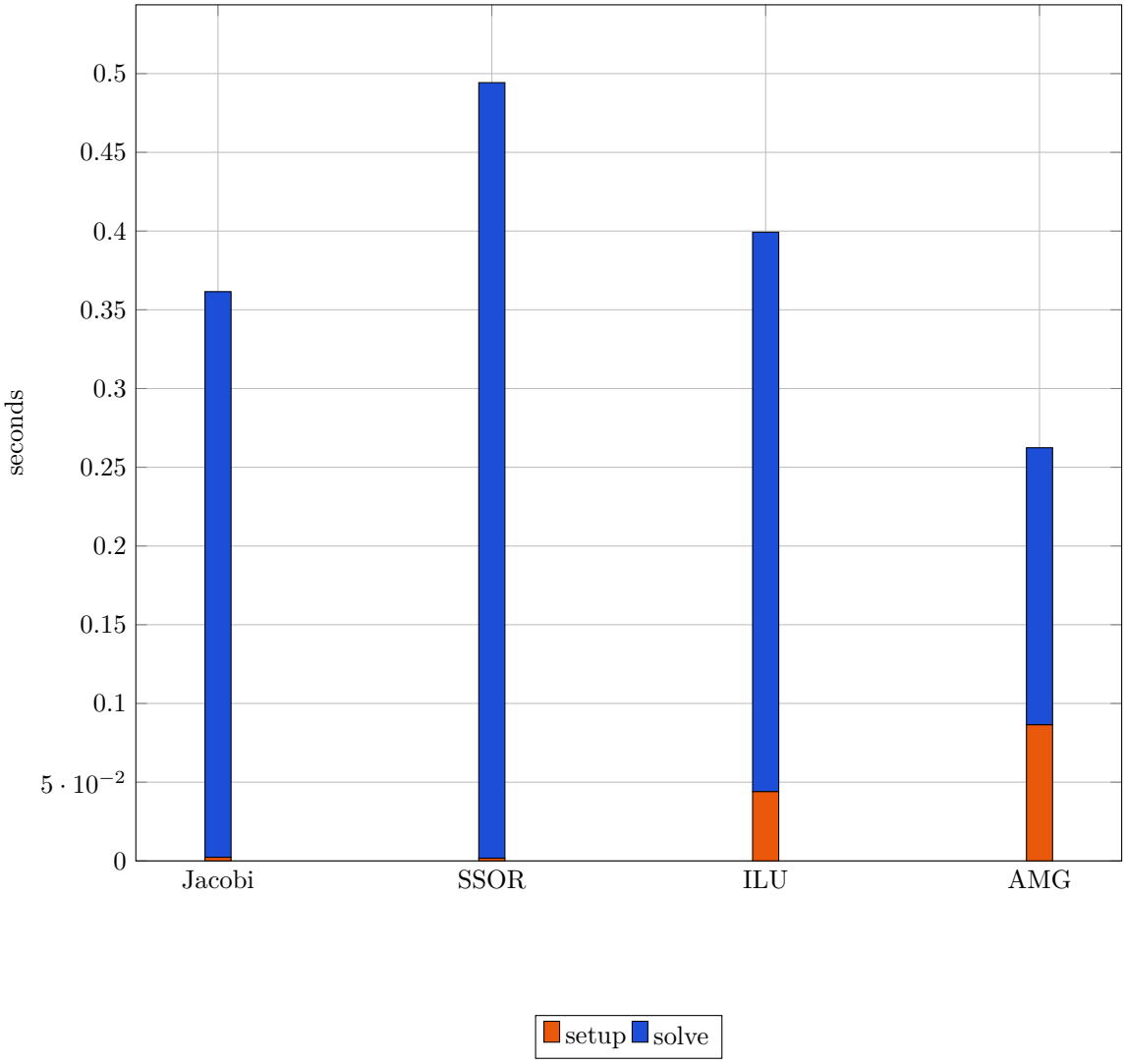


Figure 5: Cost decomposition at $p = 5$, $r = 5$, $np = 1$.

AMG spends 33% of its total in setup, ILU 11%, and Jacobi/SSOR below 1%. AMG nevertheless has the shortest solve phase because its 31 iterations are substantially more productive. SSOR is a notable counterexample to "cheap setup means cheap solve": its relaxation setup is negligible, but 211 communication- and ordering-sensitive iterations cost almost half a second.

The right metric depends on reuse. If m right-hand sides share a matrix and the preconditioner is reused, average cost becomes $t_{setup}/m + t_{solve}$. Conversely, if matrices change every step, setup is paid repeatedly. The benchmark reports both components precisely so this application-level tradeoff can be reconstructed rather than hidden in a single number.

15 Mesh dependence and algorithmic optimality

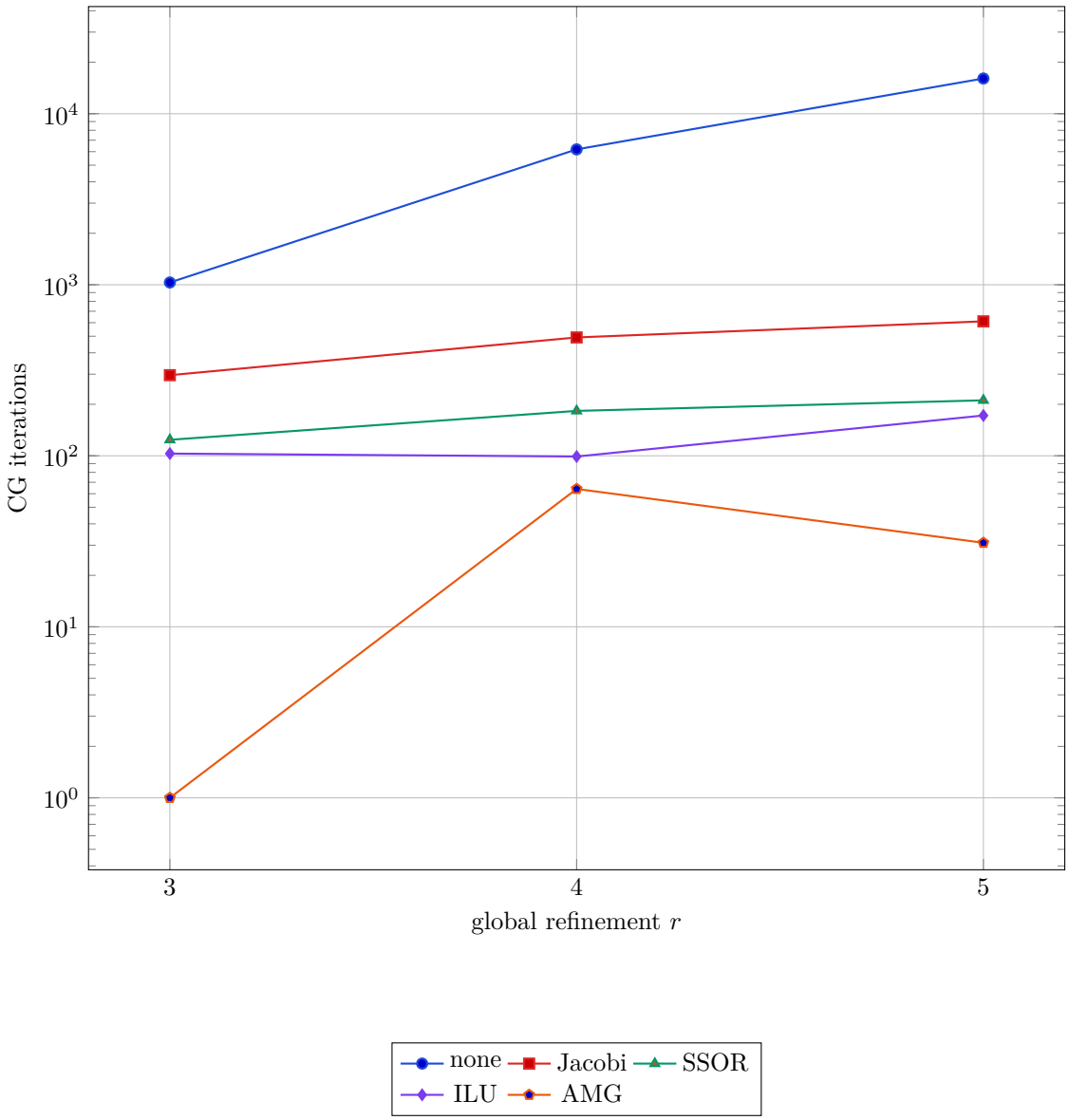


Figure 6: Iteration count versus mesh refinement for $p = 5$, $np = 1$.

From $r = 3$ to 5, DoFs grow by a factor of 49.3. Iterations grow from 1,031 to 16,067 without a preconditioner; from 296 to 611 with Jacobi; from 124 to 211 with SSOR; and from 103 to 172 with ILU. AMG rises from an anomalous one-step coarse solve to 31 iterations, with 64 iterations at $r = 4$ and 31 at $r = 5$; therefore the three-point sequence is not monotone.

Strict h-optimality would mean an iteration count bounded independently of refinement. The data do not establish that asymptotic property for any method, because only three modest meshes are tested and AMG defaults can produce coarse-grid artifacts. Still, on the two resolved meshes AMG keeps counts in the tens while the baseline reaches thousands. We describe it as the strongest evidence of practical mesh robustness, not as a proof of optimal complexity.

16 Strong scaling

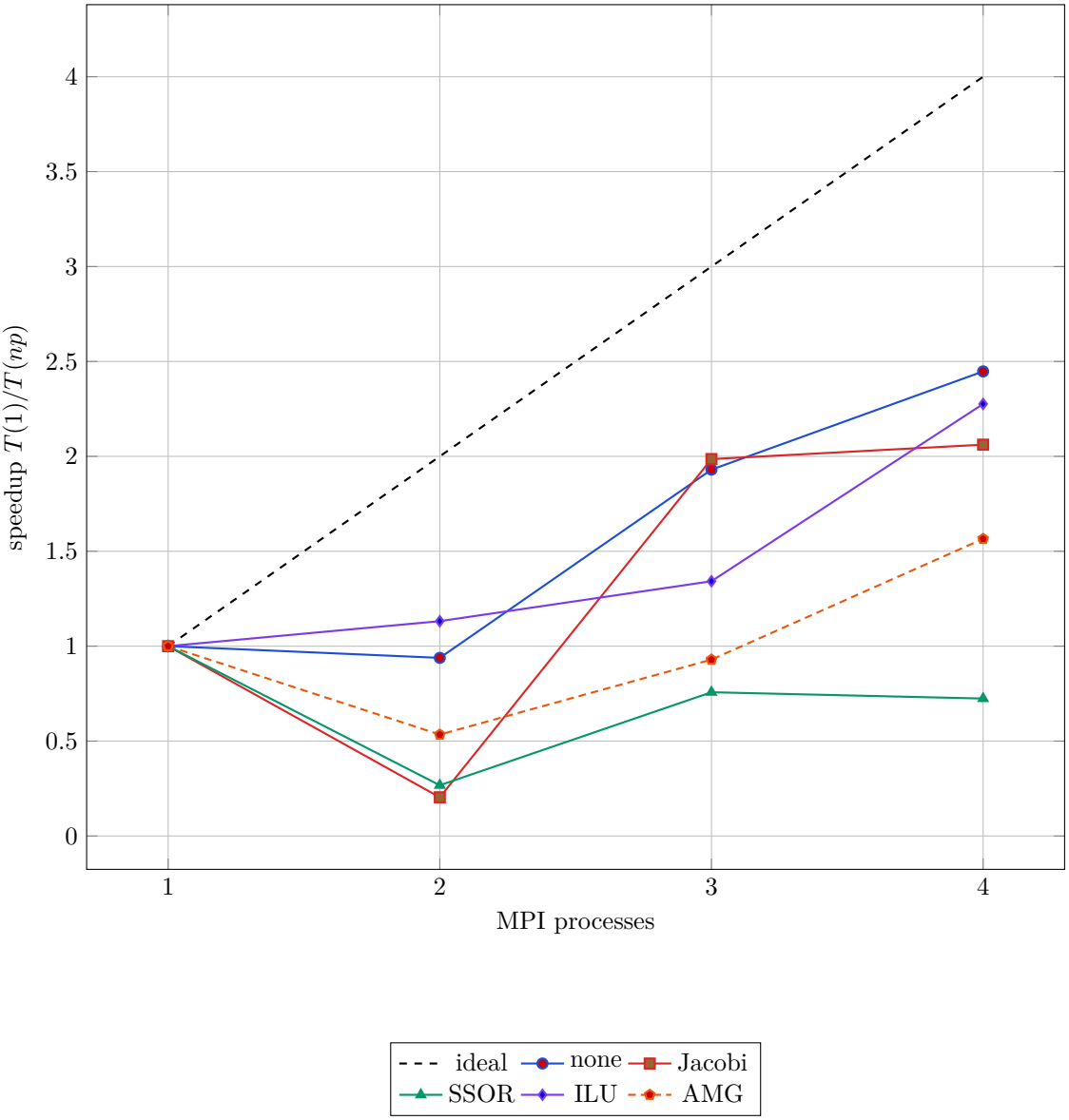


Figure 7: Strong scaling at $p = 5$, $r = 5$. Values include preconditioner setup and solve.

At four ranks the measured speedups are 2.45 (none), 2.06 (Jacobi), 0.72 (SSOR), 2.28 (ILU), and 1.57 (AMG). SSOR becomes slower, consistent with relaxation dependencies and a small per-rank workload. AMG remains the fastest absolute method at four ranks (0.168 s), narrowly ahead of Jacobi and ILU, even though its speedup is not the best.

The $np = 2$ results contain pronounced timing anomalies for several methods; $np = 3$ and 4 are often faster. This nonmonotonicity signals that single short runs do not isolate parallel performance cleanly. Repeated trials and larger meshes are needed before drawing hardware-level conclusions.

17 Quantitative synthesis

Table 4: Hardest-case summary ($p = 5$, $r = 5$). Speedup compares one and four ranks.

Method	iter. (1 rank)	$\widehat{\kappa}$	T_1 [s]	T_4 [s]	S_4
None	16,067	1.051×10^7	16.941	6.922	2.45
Jacobi	611	1.504×10^6	0.362	0.175	2.06
SSOR	211	1.701×10^5	0.494	0.683	0.72
ILU	172	1.133×10^5	0.399	0.175	2.28
AMG	31	1.847×10^1	0.262	0.168	1.57

Table 5: Decision-oriented comparison supported by the measured regime.

Method	Principal strength	Principal weakness	Appropriate use
None	zero overhead	catastrophic contrast sensitivity	baseline only
Jacobi	simple and parallel	weak spectral correction	cheap moderate cases
SSOR	fewer serial iterations	poor observed scaling	small serial/subdomain use
ILU	strong local correction	triangular dependencies	moderate rank count
AMG	best spectral robustness	hierarchy/coarse overhead	difficult elliptic systems

No single column should be read in isolation. The unpreconditioned solve has better four-rank speedup than AMG but remains 41 times slower there. SSOR has fewer iterations than Jacobi but a longer serial runtime and negative scaling. AMG wins the primary objective because it combines the lowest hard-case time with by far the smallest condition estimate and mild contrast sensitivity.

18 Discussion

Three mechanisms explain the main results. First, high contrast creates modes whose energy is localized around inclusion interfaces. Jacobi sees only nodal scale, so it cannot eliminate their collective coupling. Second, SSOR and ILU capture richer local structure, explaining their substantial serial improvement, but distributed subdomains interrupt what would otherwise be global sweeps or factors. Third, AMG explicitly transfers smooth error to coarse spaces, which attacks the global component that local methods leave behind.

The results also distinguish four meanings of "best":

- *effectiveness*: iteration and condition-number reduction;
- *efficiency*: setup plus solve wall time;
- *optimality*: bounded work growth under mesh refinement;
- *scalability*: reduced time as ranks are added to a fixed problem.

AMG leads the first two and provides the best practical evidence for the third, while its measured strong scaling is only moderate. Jacobi and the baseline expose more parallel work per solve and can therefore show respectable speedup, but speed up a much worse algorithm. The appropriate comparison is absolute time at a specified accuracy, with speedup as a secondary diagnostic.

An initially surprising feature is that AMG takes one iteration for all p on the coarsest mesh. This likely means the automatically built hierarchy or coarse solve nearly reproduces the tiny system; the missing condition estimate is consistent with an insufficient Lanczos history. Those points should not be interpreted as general one-iteration convergence. The finer meshes provide the credible robustness evidence.

19 Limitations and recommended extensions

The study is complete for the specified parameter grid, but its scope imposes limitations.

1. Each timing appears to be a single observation. Repeating cases, reporting medians and dispersion, pinning processes, and recording hardware would make performance claims stronger.
2. The largest system has only 35,937 DoFs. Strong-scaling saturation therefore arrives early; million-DoF meshes would better expose parallel asymptotics.
3. Only default preconditioner settings are used. AMG smoothers, aggregation thresholds, coarse solvers, ILU fill, overlap, and relaxation parameters deserve controlled tuning.
4. The spheres are unfitted. A mesh-aligned or adaptively refined interface study would separate geometry error from algebraic difficulty.
5. Estimated condition numbers come from CG and are unavailable for one-step cases. A small eigenvalue study on representative systems would provide independent spectral validation.

Reproducibility also benefits from preserving the raw CSV, exact source revision, deal.II/Trilinos versions, compiler flags, MPI implementation, node topology, and launch command. The repository already includes the executable modes and automation needed to regenerate the principal outputs. The most valuable next experiment is a repeated strong- and weak-scaling campaign on larger grids, followed by AMG parameter sensitivity at $p = 5$. This would test whether the current qualitative ranking persists when communication is amortized by substantial local work.

20 Conclusions

The finite-element implementation is verified: the manufactured solution achieves the theoretical P_1 rates of two in L^2 and one in H^1. The production sweep then demonstrates that coefficient heterogeneity, not merely mesh size, is the decisive algebraic challenge. Without preconditioning, the hardest system requires 16,067 CG iterations and has an estimated condition number above 10^7.

All four preconditioners help, but their quality differs. Jacobi is inexpensive and parallel but not contrast robust. SSOR and ILU make stronger local corrections; ILU is generally the more attractive of these two in total time, while SSOR scales poorly in the tested distributed setting. AMG changes the outcome qualitatively: 31 iterations, an estimated condition number of 18.5, and the smallest serial and four-rank total times for the hardest case.

Accordingly, **AMG is the recommended default for this heterogeneous elliptic problem**. That conclusion is based on absolute efficiency and spectral robustness, not on speedup alone. For very small or mildly heterogeneous systems, Jacobi can remain reasonable because setup is negligible. ILU is a competitive intermediate choice when a multigrid hierarchy is undesirable.

The broader lesson is methodological. Solver assessment must keep discretization verification, iteration robustness, setup/solve decomposition, mesh dependence, and parallel scaling distinct. The 300-case dataset supports all five views and prevents a superficially good metric from hiding a poor end-to-end method.

21 Bibliography

References

[1] A. Quarteroni, *Numerical Models for Differential Problems*, 3rd ed. Springer, 2017.

[2] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.

[3] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*. Academic Press, 2001.

[4] B. F. Smith, P. E. Bjørstad, and W. D. Gropp, *Domain Decomposition: Parallel Multilevel Methods for Elliptic PDEs*. Cambridge University Press, 1996.

[5] D. Arndt et al., "The deal.II Library, Version 9.2," *Journal of Numerical Mathematics*, 28(3), pp. 131–146, 2020.

[6] M. A. Heroux et al., "An overview of the Trilinos project," *ACM Transactions on Mathematical Software*, 31(3), pp. 397–423, 2005.

[7] NMPDE teaching staff, "Project 1: Preconditioning heterogeneous diffusion problems," Numerical Methods for Partial Differential Equations, A.Y. 2025/2026.

[8] Project source repository, `https://github.com/DOCCA0/nmpde-projects`, accessed 2026.

Data availability. The complete 300-row dataset is stored in `result/benchmark_results.csv`. The implementation and benchmark driver are included in the repository. This report was typeset directly with LaTeX; run `latexmk -pdf report.tex` to regenerate it.